

Практическая работа по Информатике

Посимвольный ввод информации. Циклы. Подпрограммы

Текст в компьютере представляется в виде последовательности чисел, каждое из которых соответствует одному символу текста. Число это называется **кодом символа**. При работе с символами для упрощения будем считать, что для хранения кода одного символа достаточно одного байта. Это соответствует кодировке ASCII и множеству её восьмибитных расширений; более того, многие программы, написанные в расчёте на однобайтовые коды символов, будут успешно работать с UTF-8.

В Паскале для работы с символами имеется специальный *символьный тип* – **char**. Значениями выражений этого типа являются одиночные символы 'a', '7', 'G', '!', ' ' (пробел), а также *специальные символы*, такие, например, как перевод строки, табуляция и т.п. В таблице эти специальные символы обозначены сочетаниями из двух и трёх заглавных латинских букв; пробел обозначен как SPC.

Таблица 1. Шестнадцатеричные коды ASCII

	.0	.1	.2	.3	.4	.5	.6	.7	.8	.9	.A	.B	.C	.D	.E	.F
0.	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1.	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2.	SPC	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3.	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4.	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5.	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6.	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7.	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

Отметим наиболее популярные специальные символы:

NUL – код 0, при выдаче на печать ничего не меняется, часто используется в качестве ограничителя строкового буфера;

BEL – bell, код 7, при выдаче на печать на экране ничего не меняется, должен прозвучать короткий звуковой сигнал;

BS – backspace, код 8, при выдаче на печать перемещает курсор на одну позицию *влево*;

HT – horizontal tabulation, табуляция, код 9, перемещает курсор вправо на позицию, кратную 8 или другому числу, установленному в настройках терминала.

LF – linefeed, код 10, перевод строки;

CR – carriage return, код 13, возврат каретки, курсор перемещается в начало текущей строки;

ESC – escape, при выводе на печать обычно означает начало управляющей кодовой последовательности, например, для перемещения курсора в заданную позицию.

В Паскале любой символ, заключённый в одинарные кавычки, обозначает сам себя. Но можно задать символ с помощью его кода в десятичной системе с префиксом #:

#10 – символ перевода строки;

#55 – '7' (\$37);

#1 ... #26 - ^A ... ^Z («кареточная» нотация);

Например, можно вместо writeln использовать write с символом перевода строки:

write('Hello, world!', #10) { или }

write('Hello, world!', ^J)

Если надо напечатать сам символ одинарной кавычки «'», то его надо удвоить, например, фраза «That's fine!» должна быть записана так:

```
writeln('That''s fine!')
```

Символы можно сравнивать так же, как и числа; при этом реально сравниваются коды этих символов: например, логическое выражение 'w' > 'd' – истинно, а '3' > 'h' – ложно. Кроме того, непрерывное расположение в ASCII-таблице символов заглавных и строчных латинских букв, а также символов других категорий, позволяет вычислением одного логического выражения определить, относится ли символ к такой категории. Например, для ответа на вопрос, «является ли символ CH строчной латинской буквой», достаточно выражения (CH >= 'a') and (CH <= 'z').

Чтобы в программе выяснить код символа, в Паскале имеется встроенная функция **ord**. Например, если у нас переменная CH имеет тип char, а переменная n – integer, то с помощью выражения присваивания n := ord(CH) в переменную n будет занесен код символа, сохранённого в CH. Обратная операция – получение символа по заданному коду – производится встроенной функцией **chr**. Выражение CH := chr(n) присвоит переменной CH символ с кодом из переменной n. Разумеется, chr(55) – это то же самое, что и #55 и '7'; но, в отличие от литералов, функция chr позволяет нам сконструировать символ, код которого мы не знали во время написания программы или который меняется во время работы программы.

Задание 1.

Допишите программу, которая печатает таблицу 2, похожую на табл. 1, только без управляющих символов:

```
program Type27;
var
  i, j: integer;
  c: char;
begin
  write('  |');                                { первая строка заголовка }
  for c := '0' to '9' do
    write('  .', c);
  for c := 'A' to 'F' do
    write('  .', c);
  writeln;
  write('  |');                                { вторая строка заголовка }
  for i := 1 to 15 do
    write('----');
  writeln;
  for i := 2 to 7 do                          { сама таблица, строка за строкой }
  begin
    write(i, ' .|');
    ...
    ...
    ...
  end
end.
```

Используйте функцию chr. В каждой строке у нас 16 символов, i - номер строки, j – номер столбца. Мы должны вычислить код очередного символа, а затем chr вернёт нам символ, соответствующий этому коду. Результатом работы программы должна быть таблица 2.

Таблица 2

	.0	.1	.2	.3	.4	.5	.6	.7	.8	.9	.A	.B	.C	.D	.E	.F
2.		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3.	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4.	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5.	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	—
6.	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7.	p	q	r	s	t	u	v	w	x	y	z	{		}	~	

Рассмотрим обратную задачу, когда нам надо получить неизвестный код символа. Напишем программу умножения двух целых чисел, которые пользователь будет вводить с клавиатуры. Зададим тип вводимых чисел как `integer`.

```
program mul2err;
var
    x,y: integer;
begin
    read(x, y);
    writeln(x*y)
end.
```

Тогда при вводе, например, букв или других неверных знаков вместо цифр будет выдано сообщение об ошибке, так как оператор `read` проверяет соответствие данных указанному типу:

```
Runtime error 106 at $000000000004001DB
$000000000004001DB
```

Вряд ли такое сообщение будет понятно пользователю, кроме того, программа на этом завершится.

Если добавить в нашу программу директиву `{$I-}` (I – input, - отключить встроенные диагностические сообщения для ошибок пользовательского ввода), то по результату вызова встроенной функции `IOResult` мы сможем узнать об ошибке внутри программы и написать об этом пользователю более корректно. Возврат этой функцией 0 означает успешный ввод, если же не ноль, то это свидетельствует об ошибке. Пример иллюстрирующего текста программы ниже.

```
program mul2err;
var
    x,y: integer;
begin
    {$I-}
    read(x, y);
    if IOResult = 0 then
        writeln(x*y)
    else
        writeln('I coldn''t parse your input')
end.
```

Теперь пользователь получит более дружественное сообщение о том, что он ошибся при вводе, чем ранее (Runtime error 106), но полностью проблемы ввода это не решает. Лучшим выходом при этом будет отказаться от услуг преобразования текста в число оператором `read`. То есть вводить информацию *посимвольно*, а затем самостоятельно разбирать введенные данные.

Так как нам потребуется анализировать ввод символов, то логичнее всего будет оформить это действие как подпрограмму.

Задание 2.

Написать программу, которая перемножает два целых числа типа `longint` и выводит результат для пользователя. Основная программа должна ждать ввода сначала первого числа, подтверждения ввода (Enter), затем второго числа с подтверждением ввода, затем выводить произведение чисел. Подпрограмма должна посимвольно проверять введенные пользователем знаки и сообщать пользователю об ошибках ввода, в случае ошибки предлагать пользователю вводить число заново. Подпрограмма должна передавать в основную программу, верно ли пользователь ввел число и само число как целое типа `longint`.